Lucas Coppock

2026-06-07

CS-255

Professor Goodman

Module 5 Project

The Role and Value of Certificate Authorities

A Certificate Authority (CA) is a trusted third party that verifies the identity of websites, organizations, and individuals before issuing digital certificates. These certificates are used to establish secure communications over the internet through protocols such as HTTPS. When a user connects to a secure website, the browser checks the certificate issued by the CA to verify that the website is authentic and that communications can be encrypted securely.

Using a CA improves security because it helps prevent man-in-the-middle attacks and website impersonation. Without a trusted CA, attackers could create fake websites that appear legitimate and trick users into entering sensitive information. A CA validates the identity of the website owner and digitally signs the certificate, allowing browsers to trust the website.

There are several advantages to using a CA. First, users can trust that they are communicating with the correct website. Second, certificates issued by trusted CAs are automatically recognized by modern web browsers, eliminating security warnings. Third, CA-issued certificates support encrypted communications that protect sensitive information such as passwords, financial records, and personal data while it is transmitted over the network.

Although CA-issued certificates are preferred for production environments, developers frequently use self-signed certificates during development and testing. Self-signed certificates provide encryption similar to CA-issued certificates, but they are not validated by a trusted third

party. As a result, web browsers display warnings because the identity of the website cannot be independently verified. Self-signed certificates are useful for development environments because they are free, easy to create, and allow developers to test HTTPS functionality before deploying applications to production.

Self-Signed Certificate Generation

For this exercise, a self-signed certificate was generated using the Java Keytool utility. The Keytool application creates a public and private key pair using the RSA encryption algorithm and stores the keys in a Java KeyStore (JKS) file. The generated certificate was then exported to a CER file and verified using the Keytool print certificate command.

The certificate was created using a 2048-bit RSA key. RSA is an asymmetric encryption algorithm that uses a public key for encryption and a private key for decryption. The 2048-bit key size is considered secure for most applications and provides strong protection against brute-force attacks.

After generating the certificate, the certificate information was successfully exported and printed using the Keytool utility. The resulting certificate contains information such as the owner, issuer, serial number, validity period, fingerprints, signature algorithm, public key algorithm, and certificate version.

The screenshots included with this submission demonstrate the successful completion of the certificate generation process and verification of the exported certificate file.

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lucas>keytool -genkey -keyalg RSA -alias selfsigned -keypass YourPassword123! -keystore keystore.jks -storepass
 YourPassword123! -validity 360 -keysize 2048
What is your first and last name?
  [Unknown]:  Lucas Coppock
What is the name of your organizational unit?
  [Unknown]:  SNHU
What is the name of your organization?
  [Unknown]:  System Design
What is the name of your City or Locality?
  [Unknown]:  Reno
What is the name of your State or Province?
  [Unknown]:  Nevada
What is the two-letter country code for this unit?
  [Unknown]:  NA
Is CN=Lucas Coppock, OU=SNHU, O=System Design, L=Reno, ST=Nevada, C=NA correct?
  [no]:  yes

Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 360 days
        for: CN=Lucas Coppock, OU=SNHU, O=System Design, L=Reno, ST=Nevada, C=NA

C:\Users\lucas>
```

```
C:\Users\lucas>keytool -export -alias selfsigned -storepass YourPassword123! -file server.cer -keystore keystore.jks
Certificate stored in file <server.cer>

C:\Users\lucas>
```

```
C:\Users\lucas>keytool -printcert -file server.cer
Owner: CN=Lucas Coppock, OU=SNHU, O=System Design, L=Reno, ST=Nevada, C=NA
Issuer: CN=Lucas Coppock, OU=SNHU, O=System Design, L=Reno, ST=Nevada, C=NA
Serial number: 2950fac79f0cc21a
Valid from: Sun Jun 07 12:30:57 PDT 2026 until: Wed Jun 02 12:30:57 PDT 2027
Certificate fingerprints:
         SHA1: A8:82:4F:C8:80:88:6B:B4:8F:49:3E:69:B5:6B:48:D6:2D:62:A6:24
         SHA256: EF:0F:EA:30:24:EE:D6:E2:E4:93:C3:AF:7C:08:33:DE:71:86:F2:A5:AD:14:EB:95:D9:4B:46:D7:A0:73:B6:79
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 57 DC 4B 09 09 F9 47 72   CB 61 5B 9D 5B BC 7E 9C  W.K...Gr.a[.[...
0010: 18 BD 76 5A                                        ..vZ
]
]


C:\Users\lucas>
```

References

Oracle. (n.d.). Keytool - Key and Certificate Management Tool. https://docs.oracle.com

Oracle. (n.d.). Java Security Standard Algorithm Names. https://docs.oracle.com

National Institute of Standards and Technology. (2020). Digital Identity Guidelines. https://www.nist.gov